

Facultad Ingeniería de Ciencias y Sistemas

Organización de Lenguajes y Compiladores 2

Sección A

Matthew Emmanuel Reyes Melgar

202202233

MANUAL USUARIO

Guatemala 19 de Marzo del año 2025

INTRODUCCIÓN

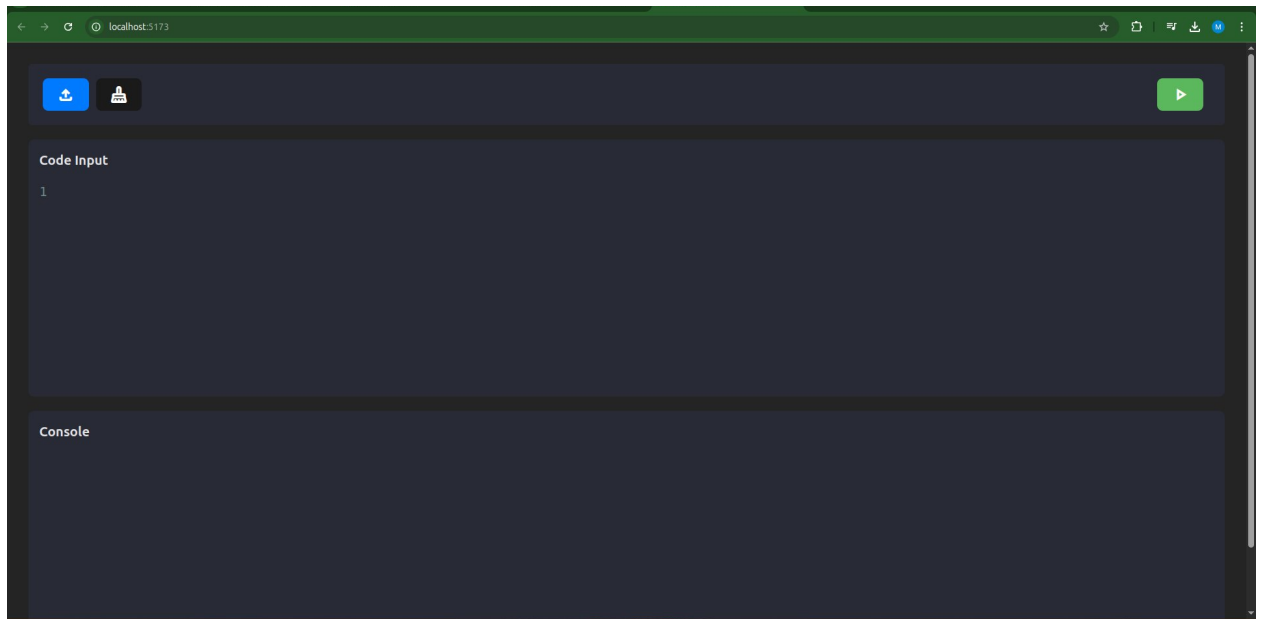
El presente documento describe el desarrollo de un compilador para el lenguaje Go, implementado mediante una API en .NET y un frontend con React Vite. Este proyecto tiene como objetivo principal proporcionar una base sólida para la construcción de un compilador funcional, aunque no implemente todas las capacidades del lenguaje Go, sí establece una estructura adecuada para su expansión y mejora.

Para el procesamiento del lenguaje se utilizó ANTLR4, una herramienta potente para el reconocimiento y procesamiento de gramáticas. Se implementó el enfoque Visitor, lo que permitió una mejor organización del código y una separación clara de los componentes gramaticales mediante etiquetas (#). Esto facilitó la estructuración de las reglas de análisis léxico, sintáctico y semántico, permitiendo una mayor modularidad y mantenimiento del código.

Además del análisis y compilación del código fuente, la API está diseñada para generar reportes de errores léxicos, sintácticos y semánticos. También ofrece funcionalidades para la creación de reportes de tabla de símbolos, proporcionando información detallada sobre los elementos identificados durante la ejecución del código.

Este proyecto representa un primer paso hacia el desarrollo de un compilador completo para Go, con una arquitectura flexible y escalable que permitirá futuras mejoras y ampliaciones en su funcionalidad.

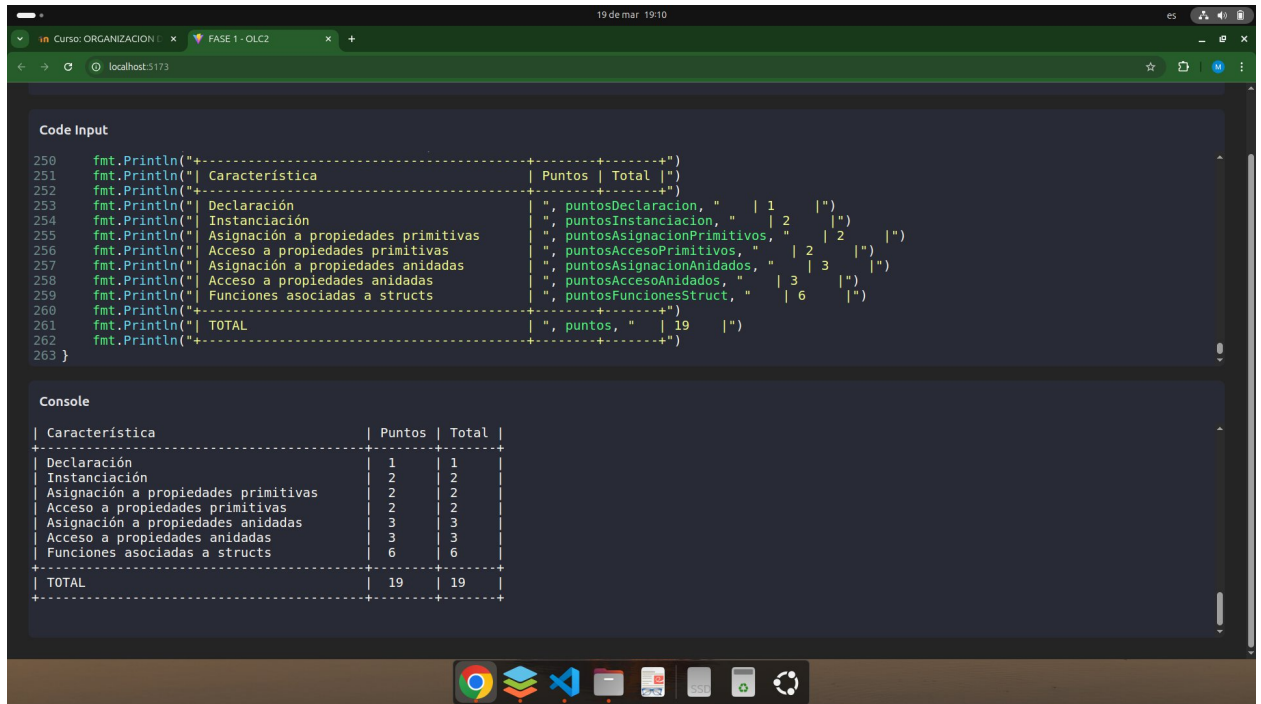
Interfaz Grafica



La interfaz consta de 3 botones, una entrada de texto y otra llamada Consola donde se podrán verificar los resultados del código ingresado al código de entrada.

1. Botón Cargar Archivo: este botón es el de color azul, posicionado en la parte izquierda superior, sirve para cargar código correspondiente a Go, para que luego puedas hacer uso el Botón de Ejecutar
2. Botón de Ejecutar Código: este botón es el de color verde, sirve para que al tener una entrada valida se puedan procesar los comandos y demás cosas, el mismo recibira respuesta y la mostrara en la parte de Console.
3. Botón de Limpiar: restaura toda la consola y la entrada para limpiar cualquier ejecución o código que puedan permanecer ahí.

EJEMPLO USO



The screenshot shows a web browser window with two tabs: "Curso: ORGANIZACION" and "FASE 1 - QLC2". The address bar shows "localhost:5173". The main content area is a code editor with the following Java code:

```
250     fmt.Println("+-----+-----+-----+")
251     fmt.Println("| Característica          | Puntos | Total |")
252     fmt.Println("+-----+-----+-----+")
253     fmt.Println("| Declaración              |", puntosDeclaracion, " | 1 |")
254     fmt.Println("| Instanciación            |", puntosInstanciacion, " | 2 |")
255     fmt.Println("| Asignación a propiedades primitivas |", puntosAsignacionPrimitivos, " | 2 |")
256     fmt.Println("| Acceso a propiedades primitivas |", puntosAccesoPrimitivos, " | 2 |")
257     fmt.Println("| Asignación a propiedades anidadas |", puntosAsignacionAnidadas, " | 3 |")
258     fmt.Println("| Acceso a propiedades anidadas |", puntosAccesoAnidadas, " | 3 |")
259     fmt.Println("| Funciones asociadas a structs |", puntosFuncionesStruct, " | 6 |")
260     fmt.Println("+-----+-----+-----+")
261     fmt.Println("| TOTAL                    |", puntos, " | 19 |")
262     fmt.Println("+-----+-----+-----+")
263 }
```

The console output shows the following table:

Característica	Puntos	Total
Declaración	1	1
Instanciación	2	2
Asignación a propiedades primitivas	2	2
Acceso a propiedades primitivas	2	2
Asignación a propiedades anidadas	3	3
Acceso a propiedades anidadas	3	3
Funciones asociadas a structs	6	6
TOTAL	19	19